

Generalità:

Nome: Lorenzo

Cognome: Mercuri

Matricola: 2145783

Corso: presenza MZ

Specifiche:

Per la **gestione del profilo** utente si è scelto di utilizzare un file (*accounts.txt*) per memorizzare le informazioni dei profili; il file è scritto in un modo specifico per poter essere letto con più semplicità tramite codice, i dati sono salvati nella forma: *nickname:password:idAvatar:partiteGiocate:partiteVinte:partitePerse:esperienza:livello*.

- L'id dell'avatar è un valore compreso tra 1 e 15, estremi inclusi, e corrisponde all'immagine scelta per il profilo. Le immagini sono situate al percorso: *JBlackJack/src/img/avatar/idAvatar.png*.
- L'esperienza non è visibile dall'utilizzatore dell'applicazione ma, viene utilizzata per stabilire l'aumento del livello tramite la formula $(150 * (\text{livello} - 1) + 50)$. Una volta saliti di livello l'esperienza per il raggiungimento di quest'ultimo viene sottratta al totale.

Sia in fase di accesso che di creazione di un nuovo profilo viene effettuato un controllo per verificare che vengano inseriti sia il nickname che la password.

The screenshot shows a login form titled "Utente" with a small spade icon. It contains two input fields: "Nickname:" with the text "admin" and "Password:" with masked characters ".....". Below the fields are two buttons: "Accedi" and "Iscriviti". At the bottom, a red error message reads: "* account già registrato".

The screenshot shows the same login form titled "Utente". The "Nickname:" and "Password:" fields are empty. Red asterisks are visible at the end of both input fields. Below the buttons, a red message reads: "* campo obbligatorio".

In fase di creazione di un nuovo profilo viene effettuato un controllo per mantenere l'unicità dei nickname, evitando la creazione di più profili con il medesimo nome.

Sempre in fase di creazione di un nuovo profilo viene controllato che la password inserita rispetti delle caratteristiche:

Utente

Nickname: admin2

Password: *

Accedi Iscriviti

* password: > 8 caratteri, maiuscola, minuscola, numero, carattere speciale

- 1) Lunghezza minima di otto caratteri;
- 2) Deve essere presente almeno una lettera maiuscola;
- 3) Deve essere presente almeno una lettera minuscola;
- 4) Deve essere presente almeno un numero;
- 5) Deve essere presente almeno un carattere speciale.

Superati i controlli si passa alla selezione dell'avatar cliccando su una delle immagini.



In fase di accesso ad un profilo vengono controllati che il nickname e la password inserita corrispondano a quelli di un profilo già esistente.

Utente

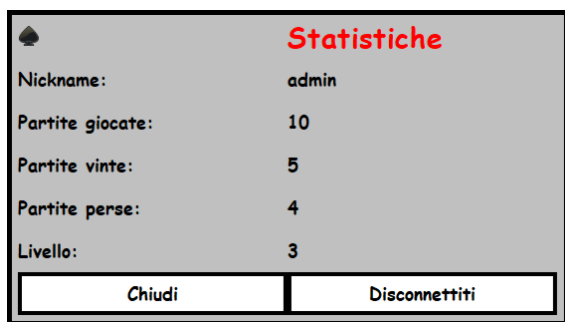
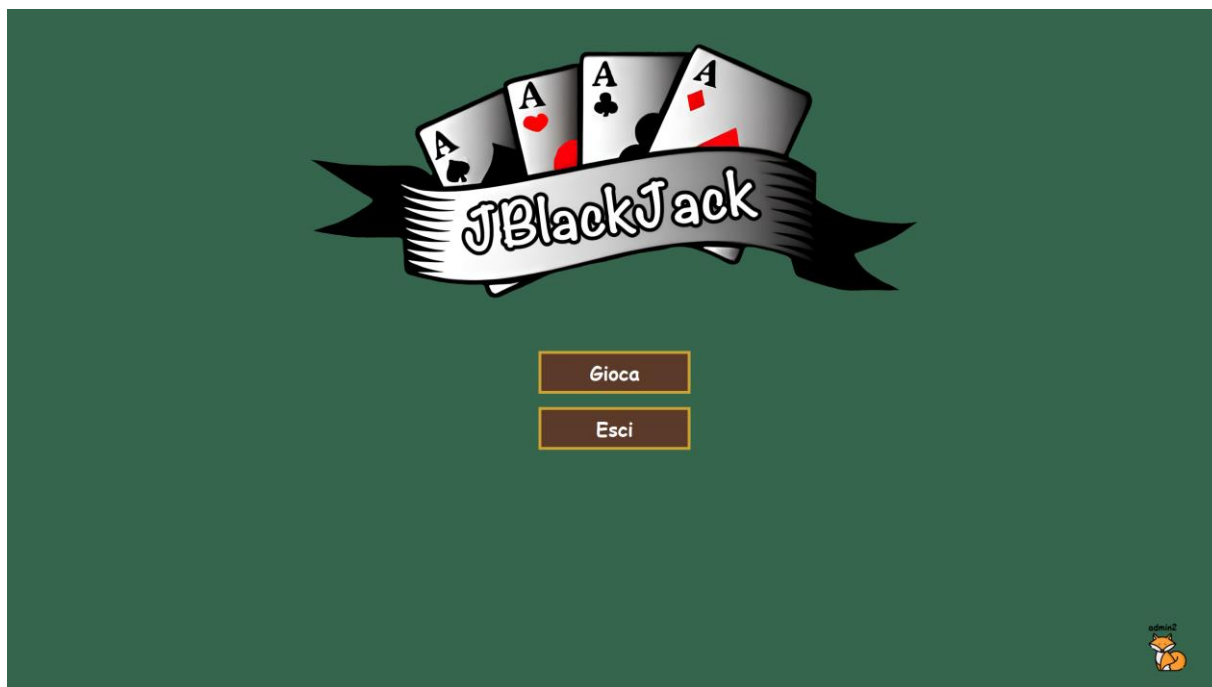
Nickname: admin

Password: *

Accedi Iscriviti

* credenziali errate

Una volta effettuato l'accesso viene visualizzata la schermata di home.



L'icona del profilo in basso a destra se cliccata, ad accesso effettuato, mostra le informazioni memorizzate del profilo altrimenti mostra la schermata per eseguire il login.

Le statistiche del profilo vengono aggiornate automaticamente una volta terminata una partita:

- aggiungendo uno al numero di partite giocate;
- aggiungendo a seconda del risultato uno al numero di partite vinte o perse;
- aggiungendo un valore di esperienza, utilizzata per salire di livello.



Nella schermata iniziale, una volta premuto il pulsante "Gioca", viene scelto il numero di giocatori artificiali della partita.

Esempio di una partita conclusa con selezione di due giocatori artificiali:



Sotto le carte del giocatore si trovano due pulsanti:

- 1) Carta, aggiunge al giocatore una carta dal mazzo;
- 2) Stai, segnale che il giocatore non ha più intenzione di pescare.

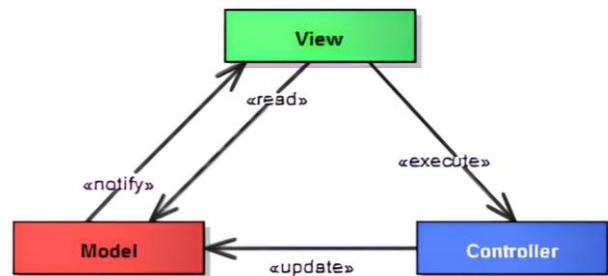
Il numero sotto le carte indica il punteggio corrente della mano sovrastante.

Se il giocatore supera il valore di ventuno, quindi ha sballato, i pulsanti vengono disabilitati automaticamente.

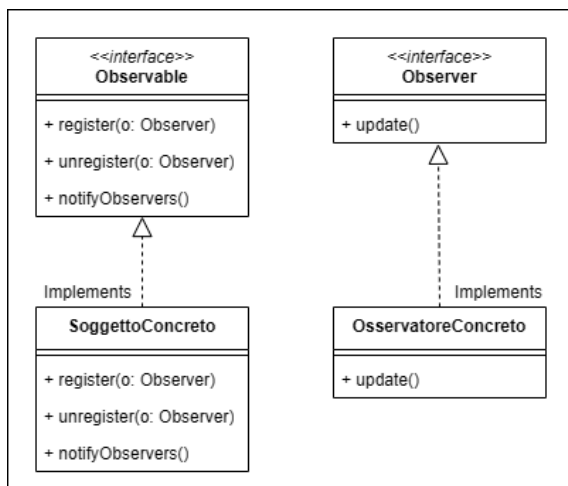
Una volta mostrato il risultato della partita appena conclusa è possibile tornare alla schermata iniziale (se si è effettuato l'accesso rimane memorizzato) cliccando il pulsante "Home", altrimenti si può chiudere l'applicazione cliccando "Esci".

Il progetto è stato realizzato seguendo un pattern ovvero un approccio standard per risolvere problemi comuni; nel seguente caso è stato utilizzato il **pattern MVC**, Model-View-Controller.

Il modello divide le tre componenti principali dell'applicazione agevolando la gestione e manutenzione del codice, consentendo inoltre la riutilizzabilità dei componenti e promuove un approccio modulare.



- 1) Il Model gestisce i dati dell'applicazione, elaborandoli e fornendo le informazioni richieste da parte degli altri componenti.
- 2) La View è passiva e non interagisce direttamente con il Model. Mostra i dati ricevuti dal Model tramite il Controller ed invia gli input dell'utente al Controller per l'elaborazione e l'aggiornamento del Model.
- 3) Il Controller è il tramite tra Model e View comunicando gli input dell'utente al Model, che notificherà del cambiamento e perciò la View riceverà tramite il Controller le nuove informazioni.



Oltre al pattern MVC è stato utilizzato anche il **pattern Observer** che stabilisce una dipendenza tra oggetti in modo che al cambio di stato del soggetto tutti i suoi osservatori vengano avvisati e aggiornati automaticamente. Il soggetto non ha bisogno di conoscere i suoi osservatori, e gli osservatori possono essere aggiunti o rimossi senza influenzare il soggetto; questa separazione favorisce un design modulare.

Il **Singleton pattern** è stato impiegato per creare una classe che possa avere un solo oggetto alla volta (un'istanza della classe). Alla prima chiamata del costruttore viene creata l'istanza, successivamente se si tenta di creare un'altra istanza la nuova punterà alla prima istanza creata; questo assicura che ci sia un controllo di accesso alle risorse.

Per realizzare il Singleton pattern è necessario che il costruttore sia privato e ci sia un metodo statico che ritorni l'istanza della classe (se non è presente la crea chiamando il costruttore). Questo pattern è stato utilizzato per avere un'unica istanza della classe principale *JBlackJack.java* e della classe *Audio Manager.java*.

```
public class AudioManager {  
    /**  
     * L'istanza singleton di {@link AudioManager}.  
     */  
    private static AudioManager instance;  
    /**  
     * Costruttore dell' {@link AudioManager} che impedisce l'istituzione diretto.  
     */  
    private AudioManager() {  
    }  
    /**  
     * Ritorna l'istanza singleton di {@link AudioManager}.  
     * Se l'istanza non esiste la crea.  
     *  
     * @return L'istanza singleton di {@link AudioManager}.  
     */  
    public static AudioManager getInstance() {  
        if (instance == null) {  
            instance = new AudioManager();  
        }  
        return instance;  
    }  
}
```

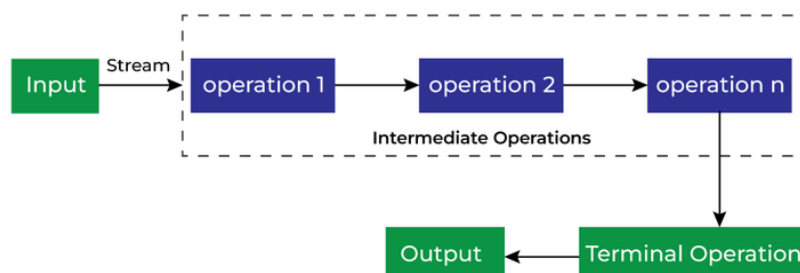
Per la parte grafica del progetto sono state utilizzate diverse librerie e classi presenti in **Java Swing** e Java AWT. È stato creato un JFrame con all'interno JLayeredPane e JPanel personalizzati contenenti JButton anch'essi modificati. Sono state anche inserite JLabel trasformate su misura con testi ed immagini. Per le personalizzazioni sono state scritte classi dedicate estendenti quelle fornite da Java Swing.

Per elaborare raccolte di dati Java mette a disposizione l'interfaccia **Stream** che supporta vari metodi che possono essere concatenati per produrre il risultato desiderato senza modificare la struttura dati originale.

Gli Stream (`Stream<T> stream;`) vengono creati a partire da una Collection di informazioni e supportano:

- operazioni intermedie, che restituiscono uno stream. Vengono eseguite in maniera pigra, cioè solo quando è stata richiesta un'operazione terminale;
- operazioni terminali, che restituiscono un risultato e terminano il flusso.

Le operazioni possono essere concatenate fino a che non si esegue un'operazione terminale.



Nel progetto una volta generato un mazzo, le cui carte sono ordinate secondo semi e valori (Asso di cuori, due di cuori, ...) si utilizza uno Stream per mescolare le carte. Nello specifico genera indici interi da zero fino al numero di carte meno uno. Per ogni carta seleziona un indice casuale, usato per determinare la carta con la quale scambiare la carta corrente. Le carte mescolate vengono raccolte in una lista, assegnata poi al mazzo, che quindi sarà mescolato.

Per l'**audio** è stata utilizzata la classe *AudioManager.java*, fornita insieme alle richieste progettuali. Alla classe sono state apportate però delle modifiche:

- le clip audio vengono ora caricate ed eliminate da una *Map<String, Clip>*;
- è stato aggiunto il metodo *stop(filename)*, che interrompe la riproduzione del file audio al percorso passato come parametro;
- è stato aggiunto il metodo *isRunning(filename)*, che ritorna lo stato di esecuzione o meno del file audio al percorso passato come parametro.

Nell'utilizzo dell'applicazione vengono riprodotti diversi audio, presenti al percorso: *JBlackJack/src/suoni/audio.wav*, ad esempio nella pagina iniziale e nella partita ci sono due colonne sonore differenti, i pulsanti quando premuti producono suoni diversi ecc.

Tutti i pulsanti al rilevamento della presenza del cursore sopra di essi si scuriscono ed una volta premuti variano il colore del loro sfondo ed il cursore si scurisce. Un'altra **animazione** presente è l'aggiunta di una carta al premere del relativo pulsante.

Altre specifiche:

Per effettuare i test sono stati creati due profili:

- 1) con nickname admin e password Admin1..., il quale è stato usato per testare l'aggiornamento delle statistiche (visibili se si accede al profilo);
- 2) con nickname Mercuri e password Lm2145783., un profilo senza partite giocate, usato solo per verificare il corretto funzionamento del salvataggio ed accesso di più profili.

Il file al percorso *JBlackJack\src\READ_ME.txt* illustra la modalità di salvataggio dei dati contenuti nel file *JBlackJack\src\accounts.txt*, memorizzante gli account.

Tutte le immagini e file audio riprodotti nel progetto sono privi di copyright, sono di pubblico dominio, scaricabili gratuitamente online.